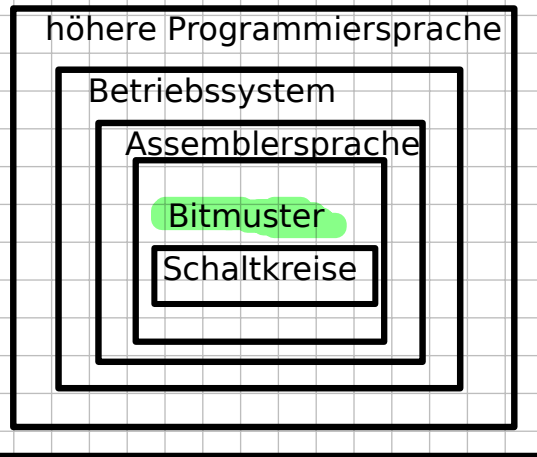




Über Bits und Bytes
oder Repräsentationen
von Symbolen, Buchstaben und Zahlen
in primitiven Datentypen

1) Zahlen und ihre Darstellungen

- b-adische Zahlendarstellung
 - Hexadezimalsystem
 - Oktalsystem
 - Binärsystem
- Konvertierung zwischen Zahlensystemen
- So rechnet der Computer mit
 - Ganzen Zahlen
 - a) das Einer-Komplement
 - b) das Zweier-Komplement
 - Fließkommazahlen
 - IEEE 754

2) Symbole, Buchstaben und andere Zeichen

- ASCII (7-Bitcodierung)
- Single-Byte & Multi-Byte „Codepage-Hölle“
- Unicode - Implementierung
UTF-8, UTF-16, UTF-32, UTF-64

$b \rightarrow$ für „Basis“
 $g \rightarrow$ für „Grundzahl“
 $\left. \begin{matrix} b \\ g \end{matrix} \right\}$ - adische Zahlendarstellung

bekannt aus Grundschule das Stellenwertsystem

des dekadischen Zahlensystems (Dezimal-System): Grundzahl „Basis ist 10“

$$\text{Zahl } z = c_0 \cdot 10^0 + c_1 \cdot 10^1 + c_2 \cdot 10^2 + \dots = \sum_{i=0}^n c_i \cdot 10^i ; c_i \in \{0, \dots, 9\}$$

z.B.: $42 = 2 \cdot 10^0 + 4 \cdot 10^1$

10 Ziffern

Allgemein: b-adische Zahlendarstellung (ganzer Zahlen)

$$\text{Zahl } z_{(B)} = \sum_{i=0}^n c_i B^i = \underbrace{(\dots((c_n \cdot B + c_{n-1}) \cdot B + c_{n-2}) \cdot B + \dots + c_1) \cdot B + c_0}_{\text{Horner-Schema der Zahl } z \text{ zur Basis } B}$$

Horner-Schema der Zahl z zur Basis B

weitere interessante Zahlensysteme in der Informatik

- se-de-adisches System, hexa-adisches System, Sederzimalsystem, Hexaderzimalsystem

$$B = 16$$

$$c_i \in \{0, \dots, 9, A, B, C, D, E, F\}$$

- okta-adisches System, Oktalsystem: $B=8, c_i \in \{0, \dots, 7\}$
- dy-adisches System, Dualsystem, Binärsystem: $B=2, c_i \in \{0, 1\}$

Konvertierung zwischen Zahlensystemen ist möglich!

Sei $N = (c_n, c_{n-1}, \dots, c_0)_{(B^*)}$ eine ganze Zahl zur Basis B^* gegeben.

Gesucht ist $\bar{N} = (\bar{c}_n, \bar{c}_{n-1}, \dots, \bar{c}_0)_{(B)}$ als Darstellung derselben Zahl zur Basis B .

Die Berechnung erfolgt im System B^* , d.h. im Ausgangssystem.

Algorithmus zur Konvertierung im Ausgangssystem von ganzen Zahlen:

Man erhält die Koeffizienten $c_n, c_{n-1}, \dots, c_0$ dadurch, daß man die Zahl N durch B dividiert, das Ergebnis als Quotient und Rest notiert, den Quotient dann wiederum durch B teilt usw., bis der Quotient verschwindet.

Die nacheinander geschriebenen Reste c_i liefern dann die Darstellung der Zahl N im Zahlensystem zur Basis B .

$$N : B = q_0 \text{ Rest } \bar{c}_0$$

$$q_0 : B = q_1 \text{ Rest } \bar{c}_1$$

...

$$q_{n-1} : B = 0 \text{ Rest } \bar{c}_n$$

Leserichtung:

$$\text{Ergebnis: } \bar{N} = \bar{c}_n \bar{c}_{n-1} \dots \bar{c}_1 \bar{c}_0$$

Beispiel: $(19)_{10} \Rightarrow \text{Dualsystem}$

$$19 : 2 = 9 \text{ R } 1$$

$$9 : 2 = 4 \text{ R } 1$$

$$4 : 2 = 2 \text{ R } 0$$

$$2 : 2 = 1 \text{ R } 0$$

$$1 : 2 = 0 \text{ R } 1$$

$$\text{Ergebnis: } (19)_{10} = (10011)_2$$

$\frac{b}{g}$ -adische Zahlendarstellung echter gebrochener Zahlen

Eine echt gebrochene Zahl r kann als Kommazahl mit m endlich vielen Stellen nach dem Komma z.B. über das folgende Hornerschema beschrieben werden:

$$\begin{aligned} r &= 0, c_{-1} c_{-2} c_{-3} \dots c_{-m} = \sum_{i=-m}^{-1} c_i B^i \\ &= \frac{1}{B} \left(c_{-1} + \frac{1}{B} \left(c_{-2} + \frac{1}{B} \left(c_{-3} + \dots \frac{1}{B} \left(c_{-m+1} + \frac{1}{B} c_{-m} \right) \dots \right) \right) \right) \end{aligned}$$

Die Konvertierung einer echt gebrochenen Zahl

r , gegeben im System zur Basis B^* , in das System zur Basis B geschieht durch sukzessive Division durch B^{-1} , d.h.

also Multiplikation mit B .

Den Vorkommateil ("Überlauf") des Ergebnisses im Multiplikationsschritt ergibt jeweils den Wert einer Nachkommastelle im Zielsystem.

Vor dem nächsten Multiplikationsschritt wird dieser Überlauf vom aktuellen Ergebnis abgezogen.

Beispiel: $(0,31)_{10}$ ins Dualesystem

$$0,31 \cdot 2 = 0,62$$

Überlauf 0 ↓ Leserichtung

$$0,62 \cdot 2 = 1,24$$

Überlauf 1

$$0,24 \cdot 2 = 0,48$$

Überlauf 0

$$0,48 \cdot 2 = 0,96$$

Überlauf 0

$$0,96 \cdot 2 = 1,92$$

Überlauf 1

$$0,92 \cdot 2 = 1,84$$

Überlauf 1

$$0,84 \cdot 2 = 1,68$$

Überlauf 1

$\vdots$

usw

$$\Rightarrow (0,31)_{10} \approx (0,0100111)_2$$

$$\text{Probe: } \frac{1}{4} + \frac{1}{32} + \frac{1}{64} + \frac{1}{128} = 0,3046875$$

b -adische Zahlendarstellung gebrochener Zahlen

Eine gebrochene Zahl, eine Zahl mit Vorkomma- und Nachkommaanteil, $r = \sum_{i=-m}^n c_i B^i$ lässt sich z.B. durch getrennte Behandlung des ganzzahligen Anteils und des echt gebrochenen Anteils konvertieren.

$$r = \sum_{i=-m}^n c_i B^i = \underbrace{\sum_{i=0}^n c_i B^i}_{\text{ganzzahliger Anteil}} + \underbrace{\sum_{i=-m}^{-1} c_i B^i}_{\text{echt gebrochener Anteil}}$$

Varianten zur Konvertierung zwischen Zahlensystemen

1. Konvertierung geschieht durch Berechnung im Ausgangssystem (wie oben) durch sukzessive Division.

2. Konvertierung geschieht durch Berechnung im Zielsystem durch sukzessive Multiplikation und Addition mittels Verwendung des Horner-Schemas (nachfolgend):

Sei $N = \sum_{i=0}^n c_i B^{*i}$ zur Basis B^* dargestellt.

Gesucht ist die Darstellung von N zur Basis B als $\bar{N} = (\bar{c}_n, \dots, \bar{c}_0)_B$

Die Berechnung geschieht im Zielsystem B , indem die Ziffern $c_0, c_1, \dots, c_n$ und B^* in das Zielsystem B konvertiert werden und in das Horner-Schema übernommen werden:

$$\begin{array}{ll} c_n \cdot B = x_1 & x_1 + c_{n-1} = x_2 \\ x_2 \cdot B = x_3 & x_3 + c_{n-2} = x_4 \\ \dots & \dots \\ x_{2(n-1)} \cdot B = x_{2n-1} & x_{2n-1} + c_0 = \bar{N} \end{array}$$

Beispiel für Konvertierungsberechnung im Zielsystem

$(2314)_{10} \Rightarrow \text{Dualsystem}$

$$\left. \begin{array}{l} B^* = (10)_{10} = (1010)_2 \\ C_3 = (2)_{10} = (10)_2 \\ C_2 = (3)_{10} = (11)_2 \\ C_1 = (1)_{10} = (1)_2 \\ C_0 = (4)_{10} = (100)_2 \end{array} \right\} \begin{array}{l} \text{Einsetzen in Hornerrechnung:} \\ (10)_2 \cdot (1010)_2 = (10100)_2 \\ (10100)_2 + (11)_2 = (10111)_2 \\ (10111)_2 \cdot (1010)_2 = (11100110)_2 \\ \begin{array}{r} 10111000 \\ + 110110 \\ \hline 11100110 \end{array} \\ (11100110)_2 + (1)_2 = (11100111)_2 \end{array}$$

$$\begin{array}{r} (11100111)_2 \cdot (1010)_2 = (100100001010)_2 \\ \begin{array}{r} 11100111000 \\ + 111111110 \\ \hline 100100001010 \end{array} \\ + \begin{array}{r} (100100000110)_2 \\ + (100)_2 \\ \hline (100100001010)_2 \end{array} \end{array}$$

Ergebnis $(2314)_{10} = (100100001010)_2$

Probe: $2^{11} + 2^8 + 2^3 + 2^1 = 2048 + 256 + 8 + 2 = 2314$

Spezielle Konvertierung zwischen Hexadezimal, Oktalsystem

Hex $\Leftrightarrow$ Bin $\Leftrightarrow$ Oktal

$$\begin{aligned} (AF4)_{16} &= (1010 \ 1111 \ 0100)_2 \quad \left. \begin{array}{l} \text{in Dreiergruppen} \\ \text{formen} \end{array} \right\} \\ &= (101 \ 011 \ 110 \ 100)_2 \\ &= (5 \ 3 \ 6 \ 4)_8 \end{aligned}$$

$$(B)_{16} = (1011)_2 \xrightarrow{\text{Dreiergruppe auffüllen}} (001 \ 011)_2 = (1 \ 3)_8$$

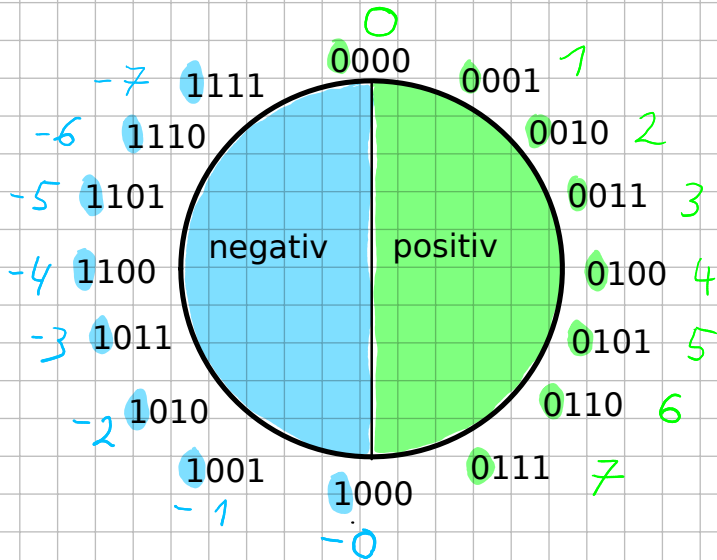
So rechnen Computer mit ganzen Zahlen:

Problem: Feste Stellenanzahl für ganze Zahlen.

Zu codieren vorzeichenbehaftete b-adische Zahl zur Basis 2.

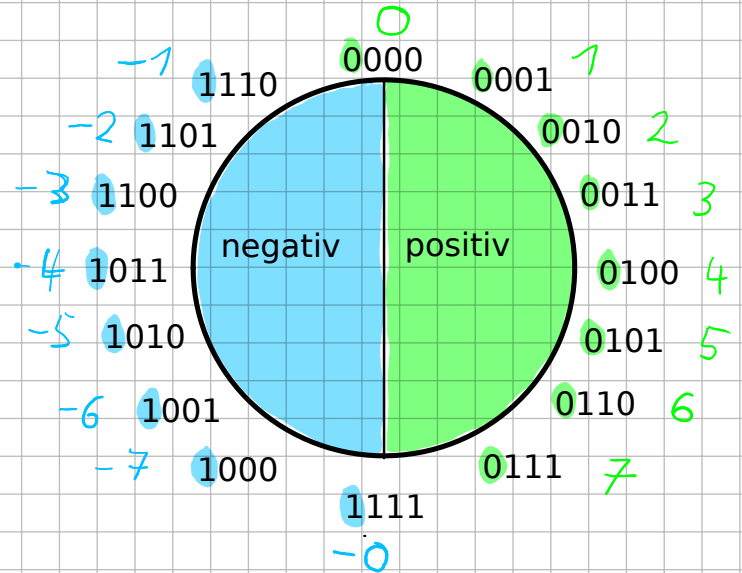
Rechnungen müssen möglich sein

1. Möglichkeit: intuitiv



Nachteil: Addition und Subtraktion benötigen gesonderte Rechenwerke auf Hardwareebene, um mit der Darstellung Zahl $z = \text{Vorzeichen} * \text{abs}(\text{Zahl})$ umzugehen.

2. Möglichkeit: (B-1)-Komplement bei Basis B=2 also Einer-Komplement



Vorteil: Subtraktion wird durch Addition durchgeführt:

$$a - b = a + (C - b) - C$$

"C - b" heißt Komplement von b zu C

C muss so gewählt werden, dass die Subtraktion einfach auszuführen ist. Bei $C = 2^n - 1$ ist dies eine bitweise Invertierung

gemeinsamer Nachteil:
doppelte Null als positive und negative Null

Rechnen im (B-1)-Komplement (Einer-Komplement) benötigt bei der Subtraktion $d = a - b$ eine Fallunterscheidung:

Fall $(a > b)$

$$a > b \Rightarrow d \geq 1 : d = a + (C - b) - C$$

$$\Leftrightarrow a + (C - b) = \underbrace{d + C}_{\geq C+1} = 2^n$$

D.h. an der $(n+1)$ -ten Stelle tritt eine nicht darstellbare 1 auf, da es nur n Stellen existieren, dies ist eine implizite Subtraktion von 2^n .

$$\Rightarrow a + (C - b) \hat{=} a + (C - b) - 2^n$$

Es müsste gelten:

$$a - b = a + (C - b) - (2^n - 1)$$

← eine 1 zuviel abgezogen, die muss dazu addiert werden

Es ergibt sich durch die Wahl von $C = B^n - 1$ bei n -Stellen bei der Berechnung $d = a - b \hat{=} a + (C - b) + 1$, wegen der impliziten Subtraktion von B^n .

Fall ($a \leq b$)

$$a \leq b \Rightarrow d \leq 0 : d = a + (C - b) - C = -|d|$$

$$\Rightarrow C - |d| = a + (C - b)$$

Beispiel 1-Komplement (Einser-Komplement) $(B-1)$ -Komplement

	$6_{10} - 4_{10}$	$3_{10} - 7_{10}$
Komplementbildung	$4 : 0100$ $-4 : 1011$	$7 : 0111$ $-7 : 1000$
Addition	$6 : 0110$ $-4 : 1011$ $+ : 10001$	$3 : 0011$ $-7 : 1000$ $+ : 1011$
Einerrücklauf	$2 : 0010$	

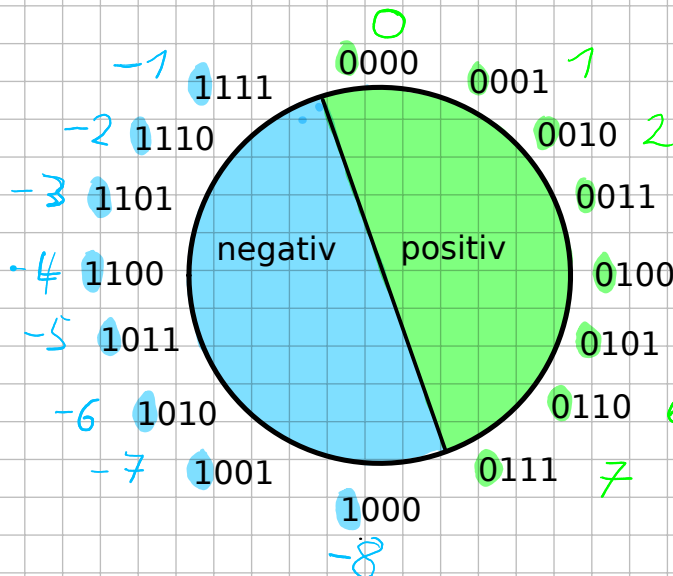
Beispiel Neuner-Komplement $(B-1)$ -Komplement bei $B=10$.

$$x = 1496 \quad y = 1 \quad d = x - y = x + (-y) - C = x + \tilde{y} - C$$

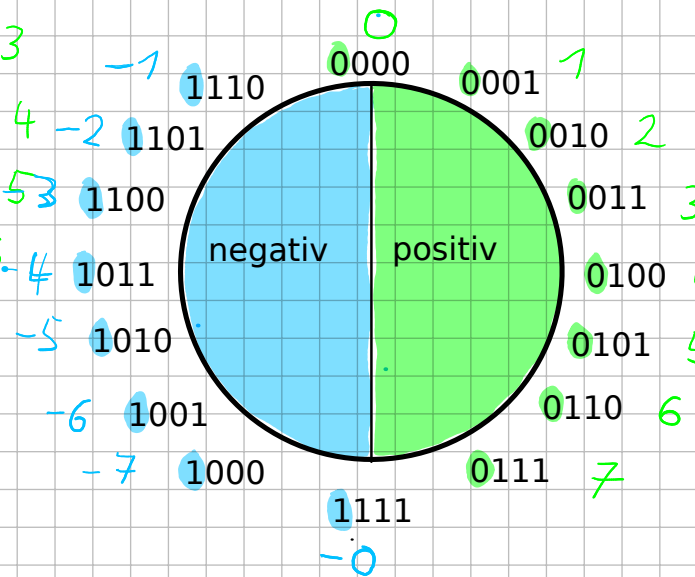
Komplementbildung	$y = 0001$ $\tilde{y} = 9998$
Addition	$x : 1496$ $\tilde{y} : 9998$ $+ : 11494$
Einerrücklauf	1495

Das Problem mit der doppelten Null lässt sich auflösen,
in dem man zulässt, dass mehr negative als positive Zahlen darstellbar sind.

3. Möglichkeit: (B)-Komplement bei Basis B=2 also Zweier-Komplement



2. Möglichkeit: (B-1)-Komplement bei Basis B=2 also Einer-Komplement



Vorteil: Die Null ist eindeutig

Auch im B-Komplement wird

die Differenz $a - b$ über eine Summation
der Komplementzahl $(C - b)$ bestimmt.

Im Einer-Komplement wurde $C = 2^n - 1$ gewählt,
für das Zweier-Komplement ist $C = 2^n = 1 \underbrace{00 \dots 0}_n$

↑ n-Nullen
Vordere 1 ist nicht mehr darstellbar.

$$a - b = a + (C - b) - C$$

$$= a + (2^n - b) - 2^n$$

wegen $(a + (2^n - b)) \bmod 2^n = (a + (2^n - b) - 2^n) \bmod 2^n$
braucht die Subtraktion von C nicht durchgeführt werden

Vergleicht man die Zahlenringe des Einer und Zweier Komplement
erkennt man das 2-Komplement einer negativen Zahl
entsteht durch das um 1 erhöhte 1-Komplement.
Ebenso fällt auf, bei der Berechnung im 2-Komplement wird keine
Fallunterscheidung nötig

Beispiel Berechnung im 2-Komplement B-Komplement

$$6_{10} - 4_{10}$$

Komplementbildung 4: 0100
 (-4), 1-Komp: 1011
 (-4), 2-Komp: 1100 } +1

Addition 6: 0110
 + (-4): 1100
 10010
 ↑
 überlauf fällt weg

Ergebnis 2: 0010

$$3_{10} - 7_{10}$$

Komplementbildung 7: 0111
 (-7), 1-Komp 1000
 (-7), 2-Komp 1001 } +1

Addition 3 0011
 + (-7) 1001
 1100

Ergebnis -4 1100

Die primitiven Datentypen von Java sind für ganze Zahlen

byte	8 bit	} jeweils vorzeichen behaftet und verwenden das B-Komplement „Zweier-Komplement“.
short	16 bit	
int	32 bit	
long	64 bit	

Binäre Operatoren für Bitmuster manipulation

Für die primitiven ganzzahligen Datentypen in Java (byte, short, int, long) sind neben den arithmetischen Rechenoperationen (+, -, *, /) und Operator zur Bestimmung des Divisionsrests % binäre Operatoren definiert, die mit dem Bitmuster, arbeiten.

Bitweises Und &, bitweises Oder |, bitweises Entweder-Oder ^

a	1	0 0 0 0 0 0 0 1
b	2	0 0 0 0 0 0 1 0
c	3	0 0 0 0 0 0 1 1
a & b	0	$\begin{array}{r} 00000001 \\ 00000010 \\ \hline 00000000 \end{array}$
a b	3	$\begin{array}{r} 00000001 \\ 00000010 \\ \hline 00000011 \end{array}$
a ^ b	3	$\begin{array}{r} 00000001 \\ 00000010 \\ \hline 00000011 \end{array}$
a ^ c	2	$\begin{array}{r} 00000001 \\ 00000011 \\ \hline 00000010 \end{array}$

Schiebe Operatoren

Left shift $a \ll 2 = (0000\ 0001) \mapsto (0000\ 0100)$
schiebe Bitmuster zwei Stellen nach links
und ergänze Lücke mit 2 Nullen.
in der Regel gilt $a \ll 1 := a \cdot 2$

Right shift, negatives (1010 1001) Bitmuster 2-Komplement.

$d \gg 2 = (1010\ 1001) \mapsto (11\ 10\ 1010)$
schiebe Bitmuster zwei Stellen nach rechts, fülle mit Vorzeichenwert auf

Right shift (zero fill) : $d \ggg 2 = (1010\ 1001) \mapsto (00\ 10\ 1010)$
schiebe Bitmuster zwei Stellen nach rechts, fülle mit Null auf

Darstellung von Fließkommazahlen, IEEE 754-Format

Festkommazahlen können im Zahlenring als n -stellige Dualzahlen bzw. Komplemente dargestellt werden, wobei dem Dezimaltrenner eine feste Stelle zugeordnet wird.

Im Gegensatz dazu sollen Fließkommazahlen kombinierbar sein, selbst wenn sie unterschiedlich vor- oder Nachkommastellen besitzen.

Eine Zahl x mit b -adischer Darstellung $x = \sum_{i=-k}^n c_i B^i$,

d.h. k Stellen im Nachkommaanteil

und n Stellen im Vorkommaanteil

wird in die Darstellung $x = m \cdot B^e$ überführt,

m ist die normalisierte Mantisse mit $\frac{1}{B} \leq m < 1$

Beispiele

$$370400 = 0,3704 \cdot 10^6$$

$$-370,4 = -0,3704 \cdot 10^3$$

$$0,003704 = 0,3704 \cdot 10^{-2}$$

$$-0,03704 = -0,3704 \cdot 10^{-1}$$

Bei Addition und Subtraktion können relevante Stellen verlorengehen, da zuerst eine Transformation auf gleichen Exponenten erforderlich ist.

$$\begin{aligned} \text{Beispiel: } & +(1,000)_2 \cdot 2^{-1} + -(1,110)_2 \cdot 2^{-2} = +(1,000)_2 \cdot 2^{-1} + -(0,111)_2 \cdot 2^{-1} \\ & = +(0,001)_2 \cdot 2^{-1} \\ & = \underline{\underline{+(1,000)_2 \cdot 2^{-4}}} \end{aligned}$$

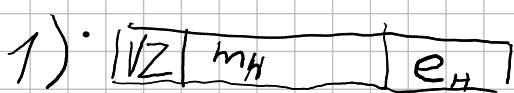
In Binärdarstellung fängt die Mantisse immer mit $\pm 0,1...$ an.
Um Speicherplatz zu sparen wird die erste 1 nach dem Komma nicht gespeichert („hidden bit“).

Aus der mathematischen Beschreibung und dem Hidden Bit ergibt sich $x = \pm m \cdot B^e = \pm 1 + m_H \cdot B^{e_H}$

Mantissenwert bei Nutzung des Hidden Bits $m_H = m \cdot (10)_2 - (1,0)_2$
Hidden Bit nicht speichern
Verschiebung der ersten Nachkommastelle vor das Komma „belegt“ Hidden Bit

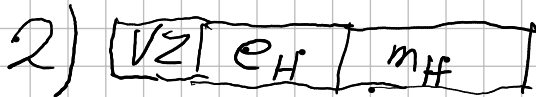
Exponentwert bei Nutzung des Hidden Bits $e_H = e - 1$
Reduzierung im Exponenten zum Ausgleich der Multiplikation beim Mantissenwert m_H

Lassen sich unterschiedliche (binäre) Darstellungen ableiten.
Beispiele:



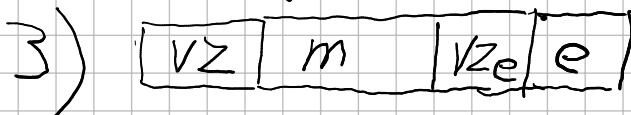
Wert des Exponenten e in Darstellung des 2-Komplements.

Wert der gespeicherten Mantisse m_H ist positiv,



VZ codiert das Vorzeichen in einem Bit für die Mantisse

Ohne „Hidden Bit“ sind auch folgende Darstellungen / Codierungen denkbar

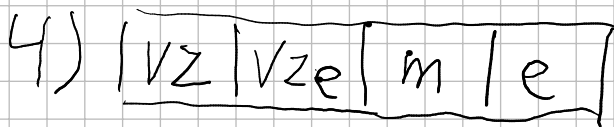


VZe Vorzeichen für Exponent

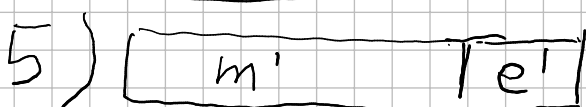
VZ Vorzeichen der Zahl

e der Absolutwert des Exponenten

m der Absolutwert der Mantisse



und weitere Abschnittspermutationen



m', e' Werte in Komplementdarstellung

Standard für Fließkommazahlen

- ANSI/IEEE Std 754-1985 ,
"Standard for Binary Floating-Point Arithmetic for microprocessor systems"
DOI: 10.1109/IEEESTD.1985.82928

- ANSI/IEEE Std 854-1987,
"Standard for Radix-Independent Floating-Point Arithmetic"
DOI: 10.1109/IEEESTD.1987.81037

- ANSI/IEEE Std 754-2019
"Standard for Floating-Point Arithmetic"
DOI: 10.1109/IEEESTD.2019.8766229

*Aktuelle fehlerbereinigte, erweiterte
Fassung als Ersatz für beide
älteren Standards.*

Definiert werden Gleitkommazahlen
mit halber (16 Bit), einfacher (32 Bit), doppelter (64 Bit)
sowie vierfacher (128 Bit) Genauigkeit.

Ergänzt werden die Grundformate durch erweiterte (extended)
und erweiterbare (extendable) Langzahl-Formate für Darstellungen von
Vielfachen von 32 Bits.

Ebenfalls neu aufgenommen wurden Datenaustauschformate.

Dicht gepackte Dezimalformate (DFP, 3 Ziffern in 10 Bit ,
so eine Dreiergruppe heißt Declet) sind ebenfalls dazugekommen.
Sie weichen von klassischen einzelzifferbasierten BCD-Formaten ab.

Speicherbild einer IEEE-754 Zahl im Datenaustauschformat.

$$\boxed{\text{VZ}} \boxed{e'} \boxed{m_H} \boxed{0} \times = \pm m \cdot B^e = (H + m_H) \cdot B^{e' - \text{Bias}} \quad \text{H das „Hidden Bit“}$$

P-1 q P-1 P-1 0

VZ ist das Vorzeichen der Zahl in einem Bit.

e' ist die Darstellung des Exponenten mit einer
stets positiven Zahl genannt „Biased Exponent“

Der wirkliche Wert des Exponenten e erhält man
durch Subtraktion eines Bias, in Abhängigkeit der Mantissenpräzision p .

e' besitzt w Bits $\text{Bias} = 2^{(w-1)} - 1$

$$e = e' - \text{Bias} \quad , \quad e_{\min} = 1 - e_{\max}$$

Werte $e' \in \{1, \dots, 2^{w-1} - 2\} \setminus \{0\}$ für normale Exponentwerte

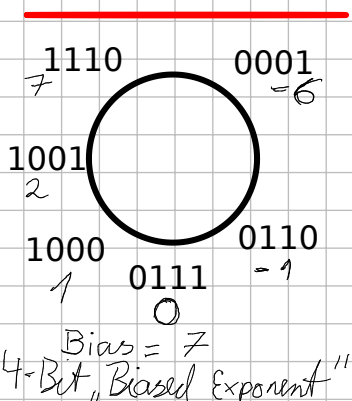
$e' = 0$ zur Darstellung von Null und anderen (subnormalen) Zahlen.

$e' = 2^w - 1$ d.h. alle w Bits gesetzt genutzt bei $\pm \infty$ und „NaN“

„NaN“ heißt „Not a Number“

NaN gibt an, dass Fehlersituationen vorliegen,
z.B.

Das Ergebnis der Division durch Null
ist ein NaN-Wert.



Am Speicherbild $\boxed{\text{VZ} \mid e' \mid m_{H'}}_{\substack{P+1 \quad P \quad P-1 \quad 0}}$ wird deutlich, dass der Wert e' um p -Stellen verschoben in dem Speicherplatz hinein codiert wird: $e' \mapsto e' \cdot 2^p$

Ist v eine IEEE-Zahl, so erhält man den Wert des Biased-Exponent durch $e' = v \cdot 2^{-p}$, also durch „Zurück schieben“ mit zero fill Rightshift.

$m_{H'}$ hat $p-1$ Bits

$m_{H'}$ stellt normalisierte Mantissen dar, wenn $e' \neq 0, e' < 2^w - 1$, dann hat das „hidden Bit“ den Wert 1.

Der Wert der dargestellten Zahl ist:

$$\begin{aligned} x &= \pm m \cdot B^e = (H + m_H) \cdot B^{e' - \text{Bias}} \\ &= (-1)^{v/2} \cdot (1 + m_{H'}) \cdot 2^{e' - \text{Bias}} \end{aligned}$$

$m_{H'}$ stellt subnormale Mantissen dar, wenn $e = 0$ gilt.

„Denormalisierte“ bzw. „subnormalisierte“ Zahlen, deren Exponentwert Null ist, sind geeignet um weitere Zahlen in der Nähe von 0 darstellen zu können, deren Absolutwert kleiner ist als der Absolutwert der kleinsten normalisierten Zahl.

Bei subnormaler Mantisse gilt, das Hidden Bit ist unbelegt.

$$\begin{aligned} x &= \pm m \cdot B^e = (H + m_H) \cdot B^{e' - \text{Bias}} \\ &= (-1)^{v/2} \cdot (0 + m_{H'}) \cdot 2^0 \end{aligned}$$

Besondere Zahlendarstellungen:

positive / negative Null mit $e' = 0$ und $m_{H'} = 0$

positiv / negativ unendlich mit $e' = 2^w - 1 = \underbrace{11 \dots 1}_w \text{ Stück}$ und $m_{H'} = 0$

NaN (undefiniertes Resultat) mit $e' = 2^w - 1 = 11 \dots 1$ und $m_{H'} \neq 0$

in m_H wird ein Grund für den „Fehler“ codiert werden; dazu zwei Arten von NaN:
„quiet NaN“, „signaling NaN“

Für Java gilt

	Vorzeichen	Exponent mit w Bits	Bias $2^{w-1}-1$	Manitisse mit $p-1$ Bits
float, 32 bit :	1 bit	8 bit	$(127)_{10}$	23 bit
double, 64 bit :	1 bit	11 bit	$(1023)_{10}$	52 bit

Unterschied zwischen Java und IEEE-754

- IEEE-754 definiert für 32 bit $2^{24}-2$ verschiedene NaN
für 64 bit $2^{53}-2$ verschiedene NaN
- Java unterscheidet in der Regel nicht zwischen NaN-Werten
- Java VM hat keine „signaling NaN“ (vgl. JVM 2.3.2, JVM 28.1)
⇒ Rechenoperationen werfen keine Exception oder Ähnliches
- Java VM unterstützt verschiedene IEEE-754 Vergleichsoperationen nicht, z.B. keine „signaling floating point comparisons“
- Seit Java 1.3 gibt es in den Wrapperklassen Float und Double die Möglichkeit manuell unterschiedliche NaN-Werte zu analysieren: (vgl. JLS 4.2.3)

• Double:

```
public static long doubleToLongBits(double value)
```

liefert long mit passendem Bitmuster für IEEE-754,
bei NaN nicht Bitmuster erhaltend
↳ Rückgabe Bitmuster für Double.NaN.

```
public static long doubleToRawLongBits(double value)
```

liefert long mit passendem Bitmuster für IEEE-754
auch bei NaN-Werten Bitmuster erhaltend

```
public static double longBitsToDouble(long bits)
```

liefert double mit gleichem Bitmuster wie im long Parameter
„signaled NaN“ werden auf andere NaN abgebildet

Es gilt:

```
value == longBitsToDouble(doubleToRawLongBits(value))  
wenn value kein signaled NaN ist.
```

• Float entsprechend

```
public static int floatToIntBits(float value)
```

```
public static int floatToRawIntBits(float value)
```

```
public static float intBitsToFloat(int bits)
```